TrafficFlowBench 2026: Team Avivengers

2026 IEEE Big Data Cup, Traffic Flow Bench. Final package report.

Team Avivengers: Yerassyl Tolegen and Mansur Yertay. Kaggle account: yetolegen. Best submission: public 0.85800, private 0.85674.

1. Summary

The benchmark has four linked tasks on synthetic freeway data: fill in blanked speed and flow readings (Task 1), predict which links will be queued in the next 30 minutes (Task 2), keep the reconstructed state physically consistent (Task 3), and recover origin-destination path flows from link counts (Task 4). The score is S_total = 0.35 S_state + 0.30 S_queue + 0.15 S_physics + 0.20 S_ODME.

Our final submission is a pipeline of separate parts, one per task, merged into one upload file:

- Task 1: a straight-line fill across each gap, then a CatBoost model that learns the error of that line from neighbouring stations, gap-edge values, the station's own readings around the target time, and time features. Trained with mask augmentation.
- Task 2: a LightGBM jam-probability model with recent-days jam-frequency features, a separate dedicated onset model (LightGBM and CatBoost), and an expected-IoU decoder that picks how many cells to flag.
- Task 3: no file of its own, so we nudge the Task 1 flow slightly toward released neighbour densities (density A) and cap it at the fundamental-diagram envelope in congested cells.
- Task 4: path flows from the organizers' reference method, fit to the link counts of the month being scored.

The best submission scored 0.85800 on the public board and 0.85674 on the private board. Across our last ten submissions the public and private scores differed by about 0.003 at most, so the method was not tuned to a single month.

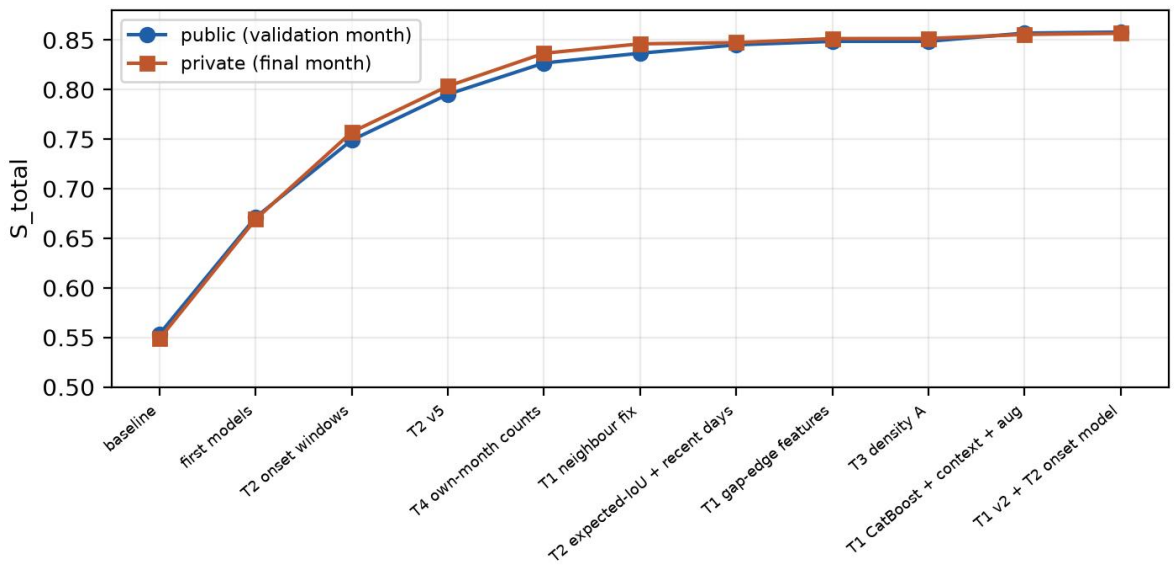


Figure 1. Score of each major submission on the public and private boards.

2. Data and how the tasks depend on each other

The data is synthetic, generated by a calibrated traffic model. There are ten directional panels (five corridors, two directions each), nine training months with answers, one validation month (public board) and one private month. The validation and private months have different incidents at different places, so anything that memorises where jams happened in one month can fail in the next. This was the main difficulty in Task 2 and shaped most of our choices.

The tasks depend on each other. Task 3 is computed from the Task 1 numbers, so a worse Task 1 hurts physics too. Task 2 starts from the observed history, and Task 4 only needs link counts. We built each task as its own notebook and merged the outputs at the end, which let us improve one task without breaking another.

3. Task 1: state reconstruction

3.1 Method

- Base fill: a straight line between the last good reading before the gap and the first good reading after it, for speed and flow separately.
- Residual model: the model predicts the error of the straight line, not the value itself. This keeps it focused on what the line misses, for example a jam starting or ending inside the gap.
- Features: the two nearest stations on each side (upstream and downstream) at the same time, their average, the difference between the target station and its neighbours, gap length, values and times at both gap edges, how much the neighbours changed between the gap edge and the target time, hour, weekday, lanes, free-flow speed and mask regime.

- Own-station context: the station's own released readings at -60, -30, -15, -10 and +10, +15, +30, +60 minutes around the target time.
- Mask augmentation: each training day borrows the real hiding pattern of a nearby day with the same regime, so readings that are released on this day but hidden on the donor day become extra training targets. This gives the model many more realistic gap shapes.
- Models: LightGBM as a fast baseline, and CatBoost (depth 8 to 10, 3000 to 6000 iterations, three seeds in the final version) trained on GPU. We scored every recipe on held-out days and used the one with the lowest error.

3.2 Results

Version	Held-out check	Leaderboard effect
Neighbour correction (task-1 v4)	reference	+0.0099 public (0.82680 to 0.83667)
Gap-edge and neighbour-change features	speed RMSE 2.34 to 2.17	+0.0035 public (0.84511 to 0.84862)
CatBoost d8 + own-station context + augmentation	relative error 0.969 vs the gap model	public 0.84867 to 0.85697, together with the Task 2 onset model
CatBoost d10, 6000 iterations, 3 seeds (task-1-v2)	relative error 0.9565 vs the gap model	+0.0010 public

Most of the gain over the older LightGBM model came from CatBoost itself and the extra context features. Gains got smaller as the model improved: the last step took a lot of GPU time for about a tenth of a point of leaderboard score.

4. Task 2: queue prediction

4.1 Why it was stuck

Task 2 stayed near 0.715 for a long time. We checked row order, keys and labels and found no bug. The cause we found in the data: between two training months, about 4.6 of the top-5 jam links are the same, but between a training month and the validation or private month only about 2.7 of 5 are the same. Jams sit in different places, so features like "this link is usually jammed at this hour" break. A "same as the last reading" baseline scores 0.2518 on validation against 0.307 on our training windows, so the scored month is also simply harder.

4.2 Method

- Target: a link is queued when speed is at or below 0.60 x free-flow speed. Windows are either onset (no queue in the history, queue in the horizon) or ongoing (a queue at T). Both count equally.
- Training windows: we rebuilt onset and ongoing windows from the training data using the rules the organizers describe, and trained LightGBM binary classifiers on per-link, per-horizon rows.
- Recent-days features: how often this link was jammed around this time over the previous 7 and 28 days, built on one timeline across train, validation and private. They only use days before the window, so the model stays causal, and they adapt to a month where jams moved.

- Expected-IoU decoder: sort the cells of a window by jam probability and flag the top k, where k maximises the expected IoU, instead of using a fixed 0.5 cutoff. For onset windows only the T+30 cells are flagged, and at least one cell always is.
- Dedicated onset model: onset windows only matter at T+30, so a separate LightGBM and CatBoost pair trained only on those cells (blended 50/50 with the shared model) learns which bottleneck breaks first instead of sharing capacity with ongoing queues.
- Causality: the award review reruns Task 2 with every row after T removed. Our features use only the history before T, the row at T, and earlier days.

4.3 Results

Change	Public leaderboard effect
Onset windows handled separately	0.67111 to 0.74901
Task 2 v5	0.79522
Expected-IoU decode and recent-days features	0.84511
Separate onset model	+0.0062 over the shared model
Ongoing-window model	-0.0009 (local gain did not transfer)
Queue-history features	-0.0003 (local gain did not transfer)

Two of our February holdout improvements (the ongoing model and the queue-history features) did not help on the public board. We kept the onset model and dropped the other two. This is the clearest case in our project of a local gain that did not carry over to a different month.

5. Task 3: physics consistency

Task 3 has no rows of its own, so the only lever is the Task 1 answer. The score is S_physics = (1/3) S_FD + (2/3) S_LWR. With the official scorer our S_FD was about 0.987. S_LWR needs hidden boundary flows, so we could not measure it locally, and we treated any change as a guess unless a proxy agreed.

The change we kept (density A): where both the reading 5 minutes before and the reading 5 minutes after a cell are released, we move the predicted density k = q / v toward those two neighbours with a 3% weight each, keep the speed, and recompute flow as k x v. In congested cells (speed below 0.58 x free speed), flow above the triangular fundamental-diagram envelope is nudged 4% toward it. The leaderboard effect was tiny (public 0.84862 to 0.84867), so we count it as safe rather than as a big win.

What did not work: a stronger version that pulled toward a linear interpolation of the released density (density B) scored worse (0.84690 against 0.84867). A direct LWR projection and gap anchoring or smoothing cost state accuracy for no clear physics gain and were dropped.

6. Task 4: ODME

The organizers release a reference path-flow method. Our first version used a file fit to the training months' link counts. The link-count term is much better when the file is fit to the counts of the month being scored, and the validation and private months each release their own link counts. Switching to the own-month counts raised the link-count score from about 0.965 to 0.9999 on validation, and the total from 0.79522 to 0.82680. We estimate S_ODME at about 0.99, so there is little left to gain there.

7. Analysis and lessons

- Month shift matters more than model size. The big jumps in Task 2 came from understanding that jam locations move between months, not from bigger models.
- Local scores can leak. Leave-own-day-out climatology leaked the label in our training windows (within-cell correlation of -0.86 in training against about 0 at inference). We still kept climatology in the final model, because removing it dropped the public score from 0.84511 to 0.83715.
- Local holdout gains do not always transfer. Of three Task 2 ideas that improved February locally, only the onset model improved the public board.
- Public and private boards agreed well. For our last ten submissions they differed by about 0.003 at most, so we did not overfit the validation month.
- The weakest part is Task 2. Our estimates by task are about 0.90 for Task 1, 0.72 to 0.75 for Task 2, 0.65 to 0.70 for Task 3 and 0.99 for Task 4. These are estimates from local checks and from the size of score jumps, because Kaggle only shows the total.

8. Final submission and reproducibility

Final selection: Kaggle submission "A v10" (sub-final v10): task-1-v2 + task-2-onset + density A + task-4. Public 0.85800, private 0.85674.

Pipeline, in order:

- task-1-v2 notebook: builds state.csv for the validation and private months.
- task-2-onset notebook: builds queue.csv.
- task-4 notebook: builds odme.csv (70,708 rows).
- sub-final notebook: applies density A to the Task 1 file, merges the three task files into one upload file, and runs the format checks (complete row sets, no blanks, queue values 0 or 1, path flows finite and non-negative).

The code package contains the numbered scripts in notebooks/ (the experiments), kaggle_pull/ (the exact Kaggle notebooks used for the final submission) and a README. The notebooks need the competition data and a GPU for the CatBoost step; CatBoost falls back to CPU, which is much slower.

9. What we would do next

- Task 2: use upstream and downstream jam spread and sudden speed drops as early-warning signals for onset windows.
- Task 3: find a way to make the Task 1 state more vehicle-conserving without hurting S_state. This needs a local approximation of S_LWR first.
- Hold out more than one month locally, so a local gain is less likely to be a one-month effect.